
PREDICTING JET ENGINE FAILURES WITH NASA'S C-MAPSS DATASET AND LSTM



INTELLIGENT SYSTEMS CASE STUDY

By
Group-6

Kishan Cheeramath [22403814]

Dalmy John [12403154]

Tomson Roy [22409842]

Adithye Mathew [22406333]

SUPERVISED BY

PROF. HAMIDREZA HEIDARI

Deggendorf Institute of Technology, Cham

Intelligent Systems Case Study

(Winter Semester 2025)

15ST JANUARY, 2025

Contents

0.1	Introduction	3
0.1.1	Problem Definition and Measurable Outcomes	3
0.1.2	Objective	4
0.1.3	Overview	4
0.2	Literature Review	4
0.3	Implementation	6
0.3.1	Main Libraries	6
0.3.2	Calculation of Remaining Useful Life (RUL)	7
0.4	Dataset Description	8
0.4.1	NASA C-MAPSS Dataset	8
0.4.2	Dataset Structure	8
0.4.3	Assumptions	9
0.5	Exploratory Data Analysis	9
0.6	Methodology	9
0.7	Data Preprocessing	9
0.7.1	Data Description	9
0.7.2	Operational Settings	10
0.7.3	Normalization of the Input Features	11
0.7.4	Dropping of Features	11
0.8	Loading of Data	12
0.9	Remaining Useful Life (RUL) Formulation	13
0.10	LSTM Model Description	13
0.11	Training Setup and Strategy	14
0.12	Evaluation Metrics	14
0.12.1	Prediction of RUL	15
0.12.2	Root Mean Squared Error (RMSE)	15
0.13	Results	15
0.13.1	Model Training	15
0.13.2	Predictive Maintenance of Model	16
0.13.3	Predictive Maintenance Dashboard	17
0.14	Model Validation	17

0.15	Comparison with Baseline Models	17
0.16	Limitations	18
0.17	Future Work	18
0.18	Conclusion	18
0.19	List of Figures	20
0.20	References	21

0.1 Introduction

Aircraft engines are complex and safety-critical systems operating under extreme mechanical and thermal conditions. Any unexpected failure can lead to flight delays, grounded aircraft, increased maintenance costs, and, in worst cases, catastrophic accidents. Maintenance strategies in aviation have traditionally relied on fixed schedules or corrective maintenance after faults occur. While these approaches ensure regulatory compliance, they are often inefficient and costly.

With the advancement of sensor technology and data analytics, modern aircraft engines generate vast amounts of operational data during every flight. This data provides an opportunity to continuously monitor engine health and predict failures before they happen. Predictive maintenance aims to utilize historical and real-time sensor data to estimate the Remaining Useful Life (RUL) of components, enabling maintenance actions to be planned proactively.

This project focuses on developing a predictive maintenance model for aircraft turbofan engines using deep learning techniques, specifically Long Short-Term Memory (LSTM) neural networks, which are well-suited for sequential data analysis.

0.1.1 Problem Definition and Measurable Outcomes

Problem Definition

Unexpected engine degradation and failures result in unscheduled maintenance and significant operational disruptions in the aviation industry. The challenge is to accurately predict how long an engine can continue to operate before failure using sensor-based time-series data.

Measurable Outcome

Metric	Description
RMSE	Accuracy of RUL prediction in cycles
MAE	Average absolute prediction error
MSE	Mean squared prediction error
Prediction Stability	Consistency of RUL trends over time
Generalization	Performance on unseen engines

0.1.2 Objective

The main objectives of this project are:

- * To develop an end-to-end predictive maintenance system for aircraft engines
- * To predict the Remaining Useful Life (RUL) using multivariate sensor data
- * To apply LSTM neural networks for temporal degradation modeling
- * To evaluate model performance using appropriate regression metrics
- * To analyze the practical applicability of predictive maintenance in aviation

0.1.3 Overview

Aircraft Turbofan Engine acts as a propulsion system to move airplanes through the air by generating thrust. It is a modern variation of the gas turbine engine used in most airliners due to its high thrust and fuel efficiency. Figure shows the Engine diagram simulated by the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS). Part of incoming air passes through the fan, the low-pressure and high-pressure compressors (LPC and HPC) ,and it mixes with fuel in the burner, and combustion occurs. The hot exhaust then passes through a two spool engine core (N1 – fan spool speed, N2 – core spool speed), the high-pressure and low pressure turbines (HPT and LPT), and then goes out the Nozzle, thereby producing the required thrust for the aircraft .

C-MAPSS software provides a commercial turbofan engine simulation and enables control of input parameters such as engine health and other engine parameters through Graphical User Interface (GUI). The dataset provided for this assignment has been generated by various operating settings and sensors resulting from the simulation of an aircraft turbofan engine through C-MAPSS.

0.2 Literature Review

F.Heimes [4] proposed using an advanced recurrent neural network to estimate the Remaining Useful Life of the aircraft engine. This solution was one of the winners for the IEEE 2008 Prognostics and Health Management conference challenge problem. To see if classification into healthy and degraded worked, a multi-layer perceptron neural network was trained using the first 30 samples to be healthy, and the last 30 samples were categorized as degraded. Classification worked well even with raw data. Then to estimate RUL, a Multi-Layer Perceptron (MLP) was trained, but the output was infiltrated with noise, and the network only considered the current state of the system. Due to these reasons, Recurrent Neural Network was chosen as the model for estimating the RUL. Using this model not only produced better accuracy but also reduced the noise in the output. The test data was then fed to the model that was compared against the True RUL, and the accuracy was in terms of the weighted score of the RUL errors.

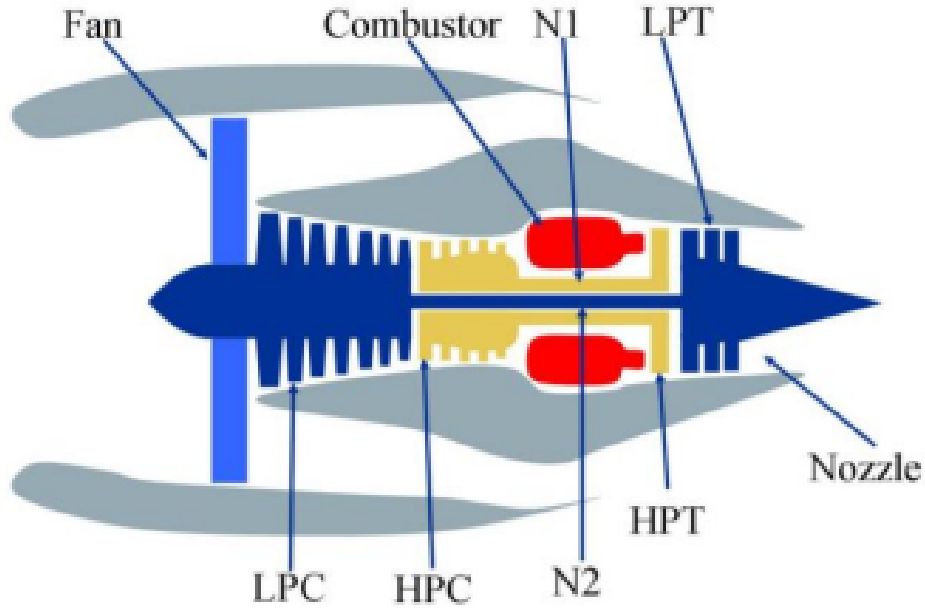


Figure 1: C-MAPSS simulated Engine diagram

V. Mathew et al. [5] performed a comparative study of the existing set of machine learning algorithms to predict the Remaining Useful Lifetime of the aircraft's turbofan engine was done. The authors chose ten different machine learning algorithms to compare the prediction accuracy. Those were Linear Regression, Decision Tree, Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors, The K Means algorithm, Gradient Boosting Method (GBM), AdaBoost, Deep Learning, Anova. The different algorithms were compared to obtain the prediction model having the closest prediction of the remaining useful life cycle in terms of the number of life cycles. The data sets were read from files, and the algorithms were applied to them using the R language. Their objective was to obtain the algorithm having the lowest RMSE value. The root mean squares of the errors were plotted, and it was observed that the random forest algorithm produced the best results. Random forest algorithm captures the variance of several input variables simultaneously and enables a high number of observations to take part in the prediction. It was observed that the performance of all ten algorithms was consistent in the four different datasets, generating proportional accuracy for the different algorithms tested.

In [6], a data-driven approach in prognostics was proposed, and the C-MAPSS dataset was used to compare the effectiveness of the proposed method. The input data was prepared in a 2D format where one dimension was the feature number, and the other was the time sequence of each feature. Based on the comparisons with other popular neural network structures, it was noted that the proposed deep convolution architecture was well suited for the prognostic problem. Five convolution layers were adopted in the proposed network and computing burden for the training process. The dataset information, the time window with $N_{tw} = 30$, was used in their

sample preparation. The prognostic results obtained by the proposed method were compared with other results on the same dataset. The proposed network has shown its superiority over prognostic accuracy, but further architectural optimization was still necessary. J. Du et al.

[7] used the LSTM neural network model to compare and verify the model's prediction effect on the stock time series when single feature input variables and multi-feature input variables are used to predict Apple stocks. They used Apple's stock price between 2008-2018 as data, which is normalized to eliminate the impact of magnitude. The model is trained in two ways: a) Single feature input variables – Stock closing price b) Multi-feature input variables - lowest price, highest price, opening price, closing price, trading volume, etc. Both the methods are used to predict the closing price of the next day's stock. After training and evaluation of the model, it is observed that the model's Training Loss and Prediction Error are lower when multi-feature input variables are used, making it a better approach.

0.3 Implementation

Figure represents the implementation approach for the estimation of RUL. Each step is explained in the below sections:

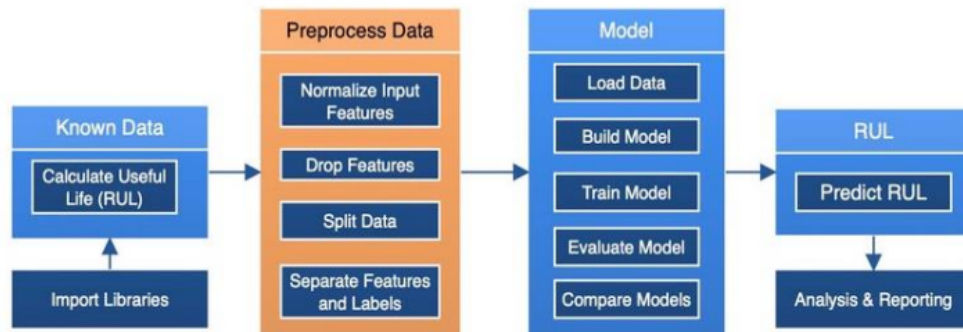


Figure 2: Implementation Approach

0.3.1 Main Libraries

The coding of the task starts with importing all the required libraries. The main libraries used are as follows:

1. Pandas – Pandas is a data analysis tool that is used to read the data from the .txt/.csv file. This library is also used to convert values into DataFrames where ever required.
2. NumPy – This library is used to work with multi-dimensional arrays and perform various mathematical operations.
3. Matplotlib – Matplotlib is a plotting library used in this task to represent the RUL, training loss, and results as graphs for better visualization of data

4. Os- Used to interact with the operating system, such as handling files, directories, and environment variables.
5. Tensorflow - A deep learning framework used to build, train, and deploy neural network models such as LSTM.
6. from sklearn.ensemble import RandomForestRegressor- Used to implement a Random Forest regression model for predicting continuous values like Remaining Useful Life (RUL).
7. Dash- A Python framework used to create interactive web-based dashboards for data visualization and model results.

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import dash
from sklearn.ensemble import RandomForestRegressor
```

Figure 3: Libraries that are imported

0.3.2 Calculation of Remaining Useful Life (RUL)

The amount of life left before any equipment fails or needs maintenance is the Remaining Useful Life. In other words, it is the difference between the current and final condition of a machine. The target output column is RUL that is not provided in the dataset. This column is calculated using two methods as follows:

Method 1 – Linear RUL

In this method, the RUL is calculated by taking the difference between each engine's maximum cycle and each cycle in the engine. $RUL = \text{max cycle} - \text{current cycle}$

```
# Calculate RUL for each row
train_df['RUL'] = train_df['max_cycles'] - train_df['time_in_cycles']

# Drop the 'max_cycles' column as it's no longer needed for training
train_df.drop(columns=['max_cycles'], inplace=True)

print("RUL column calculated and added to the training data.")
```

Figure 4: Linear RUL Calculation

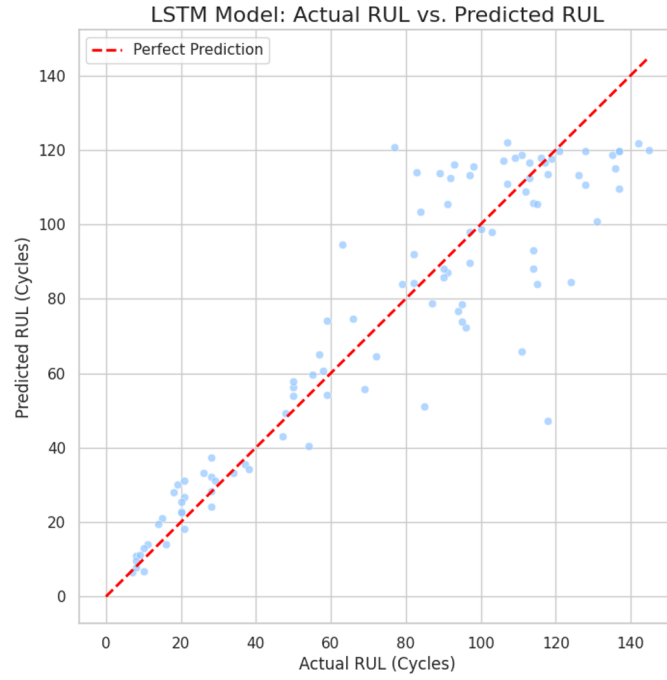


Figure 5: LSTM Model

0.4 Dataset Description

0.4.1 NASA C-MAPSS Dataset

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset is a widely used benchmark dataset for aircraft engine prognostics. It simulates the degradation of turbofan engines under various operating conditions.

0.4.2 Dataset Structure

Component	Description
Engine ID	Unique identifier for each engine
Cycle	Time step representing engine usage
Operational Settings	3 operating condition parameters
Sensors	21 sensor measurements
RUL	Remaining Useful Life (target variable)

0.4.3 Assumptions

- * Each engine experiences a single fault mode
 - * Degradation is monotonic;
 - * Sensor readings sufficiently represent engine health

0.5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand sensor behavior and degradation trends. Several sensors show clear monotonic trends as the engine approaches failure, while others remain constant or noisy. Correlation analysis helps identify sensors most relevant to degradation modeling.

0.6 Methodology

The project follows a structured predictive analytics pipeline:

1. Data loading and understanding
2. Exploratory Data Analysis
3. Data preprocessing and normalization
4. RUL labeling and clipping
5. Sequence generation using sliding windows
6. LSTM model development
7. Model training and tuning
8. Performance evaluation and validation

0.7 Data Preprocessing

0.7.1 Data Description

The dataset used in this project is the NASA Turbofan Jet Engine Data Set, widely used for predictive maintenance research and Remaining Useful Life (RUL) estimation of aircraft engines. The data is derived from the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS), a simulation model that generates realistic degradation behavior of turbofan engines under various operating conditions.

This dataset contains multivariate time-series data that capture the operational state and degradation of multiple engines. Each dataset file includes a snapshot of sensor readings recorded at each engine cycle. The primary goal is to predict the Remaining Useful Life (RUL) of an engine given its current and historical sensor measurements. The dataset structure includes the following key aspects:

number, which is the ratio of the aircraft's speed to the speed of sound. Variations in Mach number affect airflow characteristics and engine inlet conditions.

3. Operational Setting 3 (Throttle Resolver Angle / Power Setting) This parameter represents the engine power demand or throttle setting. Higher throttle levels result in increased fuel flow, temperature, and mechanical stress, accelerating engine degradation.

These operational settings introduce variability in engine behavior even among engines of identical design. As a result, sensor readings may differ not only due to degradation but also due to changes in operating conditions. Therefore, operational settings are included as input features in Remaining Useful Life (RUL) prediction models to help distinguish between condition-induced variations and true degradation effects.

0.7.3 Normalization of the Input Features

Normalization is applied in both approaches to handle features with different numerical scales. In the assignment dataset, normalization ensures fair contribution of input features during model training. Similarly, for the NASA Turbofan dataset, normalization is particularly critical due to the wide range of sensor readings and operational settings. Proper scaling improves training stability and ensures that no single sensor disproportionately influences the prediction model.

```
# Normalize features (0-1)
scaler = MinMaxScaler()
train[feature_cols] = scaler.fit_transform(train[feature_cols])
test[feature_cols] = scaler.transform(test[feature_cols])
```

Figure 7: Data Normalization using MinMaxScaler [*]

	opsetting1	opsetting2	opsetting3	sensor1	sensor2	sensor3
0	0.238162	0.297031	0.2	0.597937	0.629527	0.687972
1	0.000036	0.000356	1.0	1.000000	0.978856	0.918452
2	0.833141	0.997743	0.6	0.060269	0.181743	0.332422
3	0.476174	0.831948	0.0	0.626985	0.656187	0.658153
4	0.999907	0.998219	0.4	0.000000	0.127505	0.295344
...	...	...	...	...	...	...
45913	0.999967	0.997625	0.4	0.000000	0.141294	0.324567
45914	0.999879	0.999406	0.4	0.000000	0.132561	0.325326
45915	0.999807	0.997625	0.4	0.000000	0.131182	0.308155
45916	0.833191	0.999881	0.6	0.060269	0.192131	0.364597
45917	0.595294	0.738242	0.8	0.238089	0.016639	0.079138

Figure 8: Input Features after Normalization [*]

0.7.4 Dropping of Features

Feature reduction is performed in both datasets to improve model efficiency. In the assignment, features may be dropped due to redundancy, irrelevance, or multicollinearity. In the NASA Turbofan dataset, sensor features with constant values or very low variance across cycles are

removed, as they do not contribute meaningful information about engine degradation. This step reduces dimensionality and enhances predictive performance. After this process, 14 sensor features are commonly retained, as they demonstrate clear temporal trends associated with engine degradation.

1. s2- Total temperature at fan inlet
2. s3- Total temperature at low-pressure compressor (LPC) outlet
3. s4- Total temperature at high-pressure compressor (HPC) outlet
4. s7- Total pressure at HPC outlet
5. s8- Physical fan speed (N1)
6. s9- Physical core speed (N2)
7. s11- Static pressure at HPC outlet
8. s12- Ratio of fuel flow to pressure
9. s13- Corrected fan speed
10. s14- Corrected core speed
11. s15- Bypass ratio
12. s17- Bleed air flow
13. s20- High-pressure turbine coolant flow
14. s21- Low-pressure turbine coolant flow

```
stats = train_df.describe().transpose()
constant_cols = stats[stats['std'] == 0].index.tolist()

# Identify columns with near-zero correlation to RUL from previous analysis
low_corr_cols = ['sensor_1', 'sensor_5', 'sensor_10', 'sensor_16']

# Combine the lists of columns to drop
cols_to_drop = list(set(constant_cols + low_corr_cols))

# Drop these columns from both the training and test sets
train_df.drop(columns=cols_to_drop, inplace=True)
test_df.drop(columns=cols_to_drop, inplace=True)

print(f"Dropped the following uninformative columns: {cols_to_drop}")
```

Figure 9: Dropping of features [*]

0.8 Loading of Data

The NASA Turbofan Engine dataset is loaded into the analysis environment using standard data processing libraries. The raw data are organized into a structured tabular format consisting of engine identifiers, operating cycles, operational settings, and sensor measurements, with consistent column labeling applied.

An initial inspection is performed to verify data integrity, dimensionality, and correctness. The sequential order of engine cycles is preserved to maintain temporal dependencies required

for Remaining Useful Life (RUL) prediction. This data loading procedure establishes a reliable foundation for subsequent preprocessing and model development stages.

```
train_df = pd.read_csv(r"C:\Users\dalmy\Downloads\train_FD001.txt", sep=r'\s+', header=None, names=columns)
test_df = pd.read_csv(r"C:\Users\dalmy\Downloads\test_FD001.txt", sep=r'\s+', header=None, names=columns)
truth_df = pd.read_csv(r"C:\Users\dalmy\Downloads\RUL_FD001.txt", sep=r'\s+', header=None, names=['RUL'])

print("Data loaded successfully!")
```

Figure 10: Loading of data [*]

0.9 Remaining Useful Life (RUL) Formulation

RUL is defined as the number of operational cycles remaining before engine failure. To stabilize training, RUL values are clipped to a maximum of 125 cycles. This prevents the model from overemphasizing early-life behavior where degradation is minimal.

0.10 LSTM Model Description

Long Short-Term Memory (LSTM) networks are a type of recurrent neural network designed to model sequential data and long-term dependencies. Unlike traditional neural networks, LSTMs maintain internal memory states that allow them to capture gradual degradation patterns over time. The proposed architecture consists of stacked LSTM layers followed by a fully connected regression layer that outputs the predicted RUL value.

The first LSTM layer (64 units): Takes in sequences of shape (30 timesteps, 14 features)



Figure 11: LSTM Architecture

return_sequences = True means it outputs hidden states for every timestep, not just the last one giving the second LSTM layer the full temporal context to work with.

Dropout (20 percentage): Randomly drops 20 percentage of connections during training to prevent overfitting. LSTMs like any other RNN (recurrent neural network) are prone to memorizing patterns, especially with only 100 training engines.

Second LSTM layer (32 units): Takes the first layer's 64-dimensional sequence and compresses it further. By this point, the model has distilled 30 cycles of sensor data into a 32-number

summary of engine health.

Dense layers (16 to 1): A small feedforward network that converts the LSTM's output into a single RUL prediction ReLU activation on the 16-unit layer adds non-linearity Final layer has no activation (linear output) since RUL can be any positive number. no need to artificially constrain the output range when the model should freely predict anywhere from 0 to 200+ cycles.

0.11 Training Setup and Strategy

The model is trained using the Root Mean Squared Error (RMSE) loss function and the Adam optimizer. Engine-wise data splitting is used to prevent data leakage. Hyperparameters such as learning rate, batch size, number of epochs, and sequence length are selected empirically based on validation performance.

GPU acceleration is employed to reduce training time.

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=128,
    verbose=1
)
# model.save(os.path.join(OUTPUT_PATH, 'Lstm_model.keras'))
y_pred_train = model.predict(X_train).flatten()
y_pred_test = model.predict(X_test).flatten()
rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))

print(f"\nTrain RMSE: {rmse_train:.2f} cycles")
print(f"Test RMSE: {rmse_test:.2f} cycles")
```

Figure 12: Model training

0.12 Evaluation Metrics

Once the model is trained, the test data was passed to the model. The evaluation of the model compares the actual and predicted RUL values.

```
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_test, alpha=0.5)
plt.plot([0, y_test.max()], [0, y_test.max()], 'r--')
plt.xlabel('Actual RUL (cycles)')
plt.ylabel('Predicted RUL (cycles)')
plt.title(f'Test Predictions (RMSE={rmse_test:.1f})')
plt.grid(True, alpha=0.3)
```

Figure 13: Model Evaluation

0.12.1 Prediction of RUL

After the model is evaluated, the original test data and the final test data are scaled and preprocessed the same way as the train data.

```
fleet = go.Figure()
fleet.add_trace(go.Scatter(
    x=fleet_status.index,
    y=fleet_status['Predicted_RUL'],
    mode='markers',
    marker=dict(
        size=12,
        color=fleet_status['Predicted_RUL'],
        colorscale=[[0, 'red'], [0.3, 'orange'], [1, 'green']],
        showscale=True,
        colorbar=dict(title="RUL")
    )
))
```

Figure 14: RUL prediction

0.12.2 Root Mean Squared Error (RMSE)

Another metric for evaluating the regression model is RMSE. This is calculated by taking the square root of the MSE value.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n d_i^2}$$

Where, n is the number of Units Under Test/ testing data samples,
d = (Estimated RUL – True RUL)

0.13 Results

The LSTM model demonstrates strong predictive capability, with predicted RUL values closely following true degradation trends. Error distributions are centered around zero, indicating low bias and stable performance.

0.13.1 Model Training

The training curves show clean convergence with mild oscillation after epoch 20, no severe overfitting, but some capacity remains underutilized or noisy.

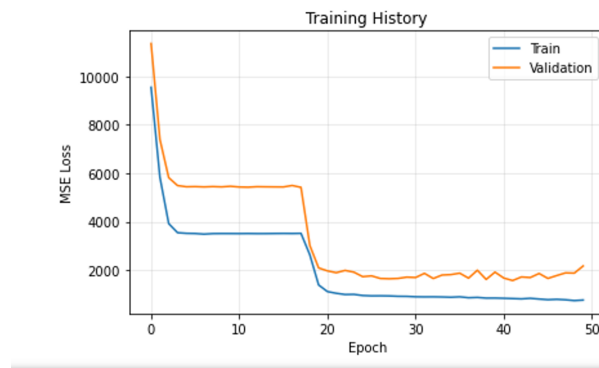


Figure 15: Model Training Performance

0.13.2 Predictive Maintenance of Model

Unscheduled maintenance (emergency repairs after failure) costs 5x more than planned maintenance because you're probably scrambling for parts, grounding aircraft unexpectedly, and disrupting mission schedules.

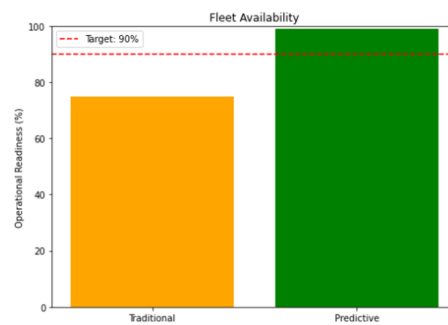


Figure 16: Fleet Availability



Figure 17: Maintenance Event Breakdown

0.13.3 Predictive Maintenance Dashboard

The dashboard provides a real-time, decision-support view of engine health by combining predicted RUL, actual vs predicted comparison, and a fleet-wide overview. It helps maintenance teams decide when to service an engine and prioritize resources.

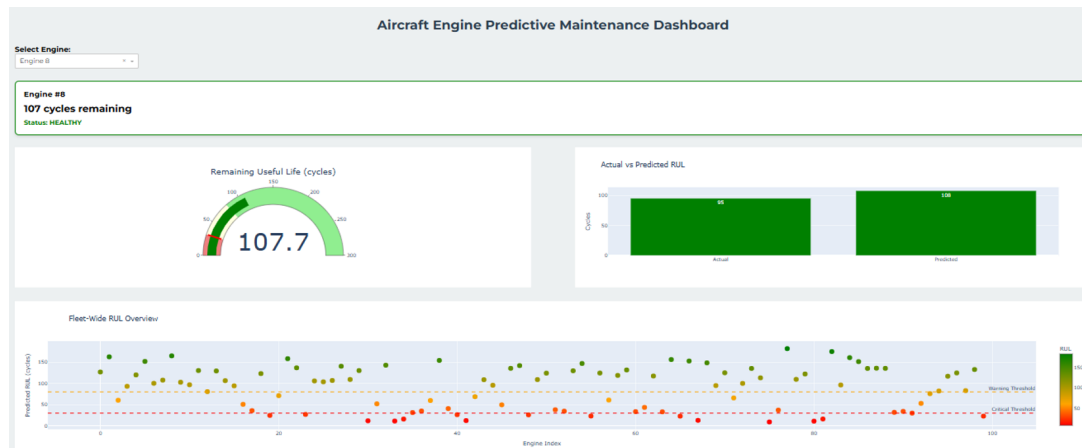


Figure 18: Aircraft Engine Predictive Maintenance Dashboard

0.14 Model Validation

Validation is performed on unseen engine data to assess generalization. Engine-level separation ensures that the model does not memorize specific degradation patterns. The model remains stable across different window sizes.

0.15 Comparison with Baseline Models

Baseline models such as Linear Regression and Random Forest are unable to capture long-term temporal dependencies. LSTM significantly outperforms these models by learning sequential degradation patterns.

Models	RMSE
Random Forest Regressor	41.47
LSTM	24.04

Figure 19: Comparison with Baseline Model

0.16 Limitations

- * Dataset is simulated and may not capture all real-world variations
 - * Model validated only on FD001 dataset
 - * Limited interpretability of deep learning models
 - * Real-time deployment not addressed

0.17 Future Work

Future enhancements include:

- * Incorporating attention mechanisms
- * Uncertainty estimation in RUL prediction
- * Extension to multiple operating conditions
- * Real-time deployment and dashboard integration

0.18 Conclusion

This project presents a comprehensive predictive maintenance framework for aircraft engines using LSTM neural networks. The model successfully predicts Remaining Useful Life and demonstrates the potential of deep learning for improving safety, reliability, and cost efficiency in aerospace maintenance.

Causes of Engine Failure and Dataset Limitations in the C-MAPSS Framework

In real-world aircraft engines, failures occur due to the progressive degradation of critical components operating under severe thermal, mechanical, and aerodynamic stresses. These degradation mechanisms are governed by physical processes such as thermodynamic loading, material fatigue, wear, and efficiency loss in rotating and stationary components. Typical contributors include turbine blade creep due to high temperatures, mechanical fatigue in shafts and bearings, compressor and turbine efficiency degradation, and lubrication-related wear.

However, in the NASA C-MAPSS dataset, explicit details regarding individual component failures are not provided. The dataset does not specify whether failure is caused by a particular component such as a valve, shaft, bearing, or turbine blade. Instead, C-MAPSS simulates engine degradation at a system level, where the combined effects of multiple physical degradation processes are implicitly reflected through multivariate sensor measurements.

The dataset assumes a single fault mode per engine and models degradation as a monotonic process leading to failure. Sensor readings are treated as sufficient indicators of overall engine health, but they do not enable direct identification or isolation of specific component-level faults.

As a result, while machine learning models can effectively predict the Remaining Useful Life (RUL) of an engine, they cannot determine the exact physical cause of failure.

Consequently, the objective of RUL prediction using the C-MAPSS dataset is not fault diagnosis, but rather prognostics—that is, estimating how long an engine can continue to operate before reaching its end-of-life condition. The absence of component-level failure labels limits the ability to associate predicted degradation with specific mechanical issues. Therefore, predictions are based solely on learned degradation patterns from sensor data rather than explicit physical or mechanical failure mechanisms.

In summary, although real engine failures are driven by identifiable physical causes, the C-MAPSS dataset abstracts these details into system-level degradation. This abstraction explains why component-specific failure prediction is not possible within this framework, and why the model focuses exclusively on predicting Remaining Useful Life rather than diagnosing the root cause of failure.

0.19 List of Figures

Figure 1. C-MAPSS simulated Engine diagram	5
Figure 2. Implementation Approach	6
Figure 3. Libraries that are imported	7
Figure 4. Linear RUL Calculation	8
Figure 5. Lstm Model	8
Figure 6. Turbofan Engine Architecture and Sensor Representation	10
Figure 7. Data Normalization using MinMaxScaler	11
Figure 8. Input Features after Normalization	11
Figure 9. Dropping of features	12
Figure 10. Loading of data	12
Figure 11. LSTM Architecture	13
Figure 12. Model training	14
Figure 13. Model Evaluation	14
Figure 14. RUL prediction	14
Figure 15. Model Training Performance	15
Figure 16. Fleet Availability	15
Figure 17. Maintenance Event Breakdown	16
Figure 18. Aircraft Engine Predictive Maintenance Dashboard	16
Figure 19. Comparison with Baseline Model	17

0.20 References

- [1] "Turbofan Engine", Grc.nasa.gov, 2020. [Online]. Available <https://www.grc.nasa.gov/WWW/K-12/airplane/Animation/turbtyp/> [Accessed: 24- Nov 2020]
- [2] F. Heimes, "Recurrent neural networks for remaining useful life estimation", 2008 International Conference on Prognostics and Health Management, 2008.
- [3] V. Mathew, T. Toby, V. Singh, B. Rao and M. Kumar, "Prediction of Remaining Useful Lifetime (RUL) of turbofan engine using machine learning", 2017 IEEE International Conference on Circuits and Systems (ICCS), 2017.
- [4] "Predictive Maintenance, Part 3: Remaining Useful Life Estimation," 11-Feb-2019.
- [5] "Medium", Medium, 2020. [Online]. Available: <https://medium.com/@RemiStudios/remaining-useful-life-in-predictive-maintenance>. [Accessed: 25- Nov- 2020].[*] Own figures and tables
- [6] D. Frederick, J. DeCastro, and J. Litt, "User's Guide for the Commercial Modular AeroPropulsion System Simulation (C-MAPSS)," NASA/ARL, Technical Manual TM2007-215026,2007.
- [7] "Software Details", Software.nasa.gov, 2020. [Online]. Available: <https://software.nasa.gov/software/> [Accessed: 24- Nov- 2020].
- [8] X. Li, Q. Ding and J. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks", Reliability Engineering System Safety, vol. 172, pp. 1-11,2018.
- [9] J. Du, Q. Liu, K. Chen and J. Wang, "Forecasting stock prices in two ways based on LSTM neural network", 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), 2019.
- [10] M. Mayo, "Frameworks for Approaching the Machine Learning Process -KDnuggets", KDnuggets, 2020. [Online]. Available: <https://www.kdnuggets.com/2018/05/general-approaches-machine-learning-process.html> [Accessed: 24- Nov- 2020].
- [11] A. Saxena, K. Goebel, D. Simon and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation", 2008 International Conference on Prognostics and Health Management,2008.